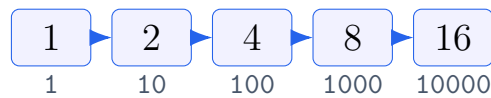


Checking Whether a Number Is a Power of Two

A beginner-friendly guide to logic, patterns, and clean problem solving



What you will learn

- How to translate a math pattern into code.
- How to reason through edge cases before writing a solution.
- Why the bit manipulation trick works, not just how to memorize it.
- How to compare solutions using time and space complexity.

1. Problem in One Sentence

Given an integer n , decide whether it belongs to the sequence

$$1, 2, 4, 8, 16, 32, 64, \dots$$

In mathematical form, n is a power of two when

$$n = 2^k \quad \text{for some integer } k \geq 0.$$

Beginner mindset

Before coding, ask: *What special pattern do valid answers have?* Here, valid numbers keep dividing evenly by 2 until they become 1.

2. First Observations

Number	Reasoning	Answer
1	$1 = 2^0$, so it is included.	True
2	$2 = 2^1$.	True
8	$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.	True
12	$12 \rightarrow 6 \rightarrow 3$, and 3 is not 1.	False
0	No nonnegative exponent gives $2^k = 0$.	False
-4	Powers of two in this problem are positive.	False

3. Approach 1: Iterative Division

Core Idea

If n is positive and even, keep dividing by 2. At the end, only a true power of two will land exactly on 1.

Thought process

A number like 16 is built by multiplying 1 by 2 repeatedly. So we can reverse that process by dividing by 2 repeatedly.

```
1 def isPowerOfTwo(n: int) -> bool:
2     if n <= 0:
3         return False
4
5     while n % 2 == 0:
6         n = n // 2
7
8     return n == 1
```

Trace Example



Because the process ends at 1, the result is **True**.

For 12:

$$12 \rightarrow 6 \rightarrow 3$$

The process stops at 3, so the result is **False**.

Why the Guard Clause Matters

The check `if n <= 0` prevents two problems:

- $n = 0$ would loop forever because $0\%2 = 0$ and $0//2 = 0$.
- Negative numbers are not valid powers of two for this problem.

Complexity

Each iteration cuts the number in half, so the number of iterations is proportional to $\log_2 n$.

$$\text{Time: } O(\log n) \quad \text{Space: } O(1)$$

4. Approach 2: Recursive Division

Core Idea

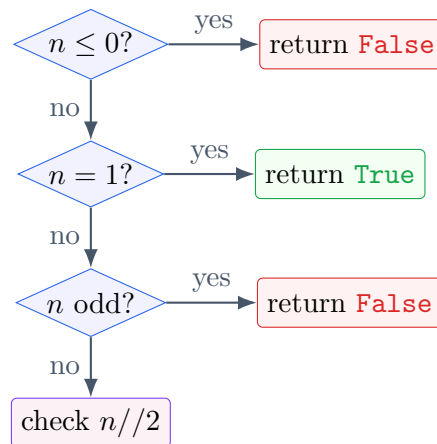
Recursion uses the same logic as iteration, but expresses it as a smaller version of the same problem.

```
1 def isPowerOfTwo(n: int) -> bool:
2     if n <= 0:
3         return False
4     if n == 1:
5         return True
6     if n % 2 != 0:
7         return False
8
9     return isPowerOfTwo(n // 2)
```

Recursive thinking

To solve `isPowerOfTwo(16)`, ask a smaller question: `isPowerOfTwo(8)`. Keep shrinking the input until a base case gives the answer.

Decision Flow



Complexity

The value is still divided by 2 each time.

Time: $O(\log n)$ Space: $O(\log n)$

The extra space comes from the recursive call stack.

5. Best Approach: Bit Manipulation

The Pattern

A positive power of two has exactly one 1 bit in binary.

$$1 = 1_2, \quad 2 = 10_2, \quad 4 = 100_2, \quad 8 = 1000_2, \quad 16 = 10000_2$$

When subtracting 1 from a power of two, that single 1 bit turns into 0, and all bits to its right become 1:

$$8 = 1000_2, \quad 7 = 0111_2$$

These two binary numbers share no position where both have a 1. Therefore:

$$8 \& 7 = 0$$

n	Binary n	Binary $n - 1$	$n \& (n - 1)$
8	1000	0111	0000
16	10000	01111	00000
12	1100	1011	1000

```
1 def isPowerOfTwo(n: int) -> bool:
2     return n > 0 and (n & (n - 1)) == 0
```

Why this is the cleanest solution

The expression `n & (n - 1)` removes the lowest set bit of `n`. A power of two has only one set bit, so removing it leaves zero.

Complexity

For typical fixed-width integer arithmetic, the bitwise check is constant time and constant space.

Time: $O(1)$ Space: $O(1)$

6. How to Build the Solution Yourself

Problem-solving checklist

1. **Clarify valid input:** Is 0 allowed? Are negative numbers valid?
2. **List examples:** Compare 1, 2, 4, 8 with 3, 6, 12.
3. **Find the pattern:** Powers of two divide evenly by 2 until 1.
4. **Write a safe basic solution:** Use iterative division with an edge-case guard.
5. **Optimize using structure:** Notice the one-set-bit binary pattern.

7. Comparison

Approach	Time	Space	Best for
Iterative division	$O(\log n)$	$O(1)$	Learning the core mathematical idea.
Recursive division	$O(\log n)$	$O(\log n)$	Practicing base cases and recursive thinking.
Bit manipulation	$O(1)$	$O(1)$	Writing the shortest and most efficient solution.

8. Final Recommendation

Use the bit manipulation solution in interviews or production-style code:

```
1 def isPowerOfTwo(n: int) -> bool:
2     return n > 0 and (n & (n - 1)) == 0
```

But if you are learning, first understand the division method. The bit trick is powerful because it is just the same idea viewed through binary: powers of two have one active bit, and `n & (n - 1)` tests exactly that property.